DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES · MLOps

TP 10 — Construire, déployer et sécuriser l'API REST du modèle

Durée : 3 h 30 · Partie 8 du cours · binôme de 2

Ce TP met en pratique les trois points de la partie 8 du cours, entièrement sur votre machine : aucun hébergeur, aucun compte en ligne. Partie A : vous comparez Flask et FastAPI sur le même service. Partie B : vous construisez l'API de détection de fraude en suivant les six étapes du cours, vous la testez et vous la servez comme en production. Partie C : vous jouez les deux rôles, attaquant puis défenseur. Un « brouillon » volontairement vulnérable et la version sécurisée du cours tournent côte à côte ; pour chacune des 8 bonnes pratiques, vous lancez la même attaque sur les deux et vous constatez ce qui la bloque. Fil rouge : le dossier de Mme Benali doit toujours donner une probabilité de 0,8899.

CE QUI CHANGE DANS CETTE VERSION
Tout se fait en local : plus de registre en ligne ni d'hébergeur (Render, etc.). Le TP suit le P08 point par point, avec les mêmes fichiers et les mêmes routes (/health, /v1/model-info, /v1/predict). Nouveau : une partie C « attaque puis protection » pour les 8 pratiques de sécurité du cours. Les requêtes passent par un petit script Python, identique sous Linux, macOS et Windows. Toutes les sorties ont été obtenues réellement.

Objectifs pédagogiques

- Expliquer ce que fait une API autour d'un modèle : requête, réponse, JSON, codes HTTP
- Comparer Flask et FastAPI sur une entrée invalide (500 contre 422)
- Construire l'API en six étapes : chargement unique, schéma d'entrée, schéma de sortie, routes versionnées, journal, tests
- Mesurer le coût d'un modèle rechargé à chaque requête et le gain de plusieurs processus
- Mener 8 attaques sur une API vulnérable et vérifier que chaque bonne pratique les bloque : HTTPS, clé d'API, secrets, validation, limite de débit, erreurs discrètes, conteneur sans privilèges, journal
- Conteneuriser la version sécurisée sans aucun secret dans l'image

Prérequis

- Le projet `tp-mlops` et son environnement `.venv`, avec `models/model.pkl` (forêt du TP 3, scikit-learn 1.9.1)
- Docker installé et démarré (TP 9) : il ne sert qu'aux étapes 11 et 15
- Le cours P08 sous les yeux, en particulier la checklist de sécurité
- Prévoir 4 terminaux ouverts dans le dossier du TP, chacun avec l'environnement activé

Installation

Les bibliothèques du modèle (scikit-learn, pandas, joblib) sont déjà installées. On ajoute les deux cadres web, le serveur, les outils de test, `trustme` (crée un certificat HTTPS local) et `python-dotenv` (lit le fichier de clé).

terminal

```
cd tp-mlops && source .venv/bin/activate
pip install fastapi uvicorn flask requests pytest httpx2 trustme python-dotenv
```

RÉSULTAT ATTENDU

```
Successfully installed ... fastapi-0.162.2 flask-3.1.3 httpx2-2.16.0 pytest-9.2.0
python-dotenv-1.2.4 requests-2.34.2 trustme-1.2.1 uvicorn-0.54.0 ...
(versions egales ou plus recentes)
```

Déroulement

Étape 1 — Partie A · Préparer le dossier du TP (5 min)

Le TP se déroule dans un sous-dossier `tp10-api`, avec une copie du modèle. Toutes les commandes se lancent depuis ce dossier.

terminal

```
mkdir -p tp10-api/api tp10-api/models tp10-api/tests tp10-api/attaques
cp models/model.pkl tp10-api/models/
cd tp10-api
```

Le premier fichier est un petit client, `appel.py`. Il envoie une requête et affiche le code HTTP puis la réponse. Il envoie le dossier de Mme Benali ; `champ=valeur` modifie un champ et `--cle` ajoute la clé d'API. On l'utilise à la place de curl : il marche de la même façon sous Linux, macOS et Windows.

appel.py

```
cat > appel.py << 'EOF'
"""Appelle l'API et affiche le code HTTP puis la reponse.
Exemples :
  python appel.py GET  http://localhost:8000/health
  python appel.py POST http://localhost:8000/v1/predict montant=abc
  python appel.py POST https://localhost:8443/v1/predict --cle MA_CLE
Le corps envoye est le dossier de Mme Benali ; champ=valeur le modifie."""
import sys

import requests

methode, url, args = sys.argv[1].upper(), sys.argv[2], sys.argv[3:]
entetes = {}
if "--cle" in args:                                  # cle d'API dans l'en-tete X-API-Key
    i = args.index("--cle")
    entetes["X-API-Key"] = args[i + 1]
    args = args[:i] + args[i + 2:]


def valeur(texte):                                   # "1800" -> 1800, "abc" -> "abc"
    for conversion in (int, float):
        try:
            return conversion(texte)
        except ValueError:
            pass
    return texte


corps = {"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}
for a in args:
    champ, texte = a.split("=", 1)
    corps[champ] = valeur(texte)

r = requests.request(methode, url, headers=entetes, timeout=10, verify="certs/client.pem",
                     json=corps if methode == "POST" else None)
print(r.status_code, r.reason)
print(r.text[:400])
EOF
```

Si ça coince : tous les fichiers du TP sont écrits sans accents : sous Windows, l'écriture en `-Encoding ascii` remplacerait les lettres accentuées par des « ? ».

Étape 2 — Le même service avec Flask et avec FastAPI (15 min)

Rappel du cours : les deux cadres font la même chose, mais FastAPI vérifie les données reçues à partir d'un schéma. Écrivez les deux versions, lancez-les chacune dans un terminal, puis envoyez un dossier correct et un dossier avec `montant=abc` à chacune.

flask_app.py

```
cat > flask_app.py << 'EOF'
import joblib
import pandas as pd
from flask import Flask, jsonify, request

app = Flask(__name__)
```

```
modele = joblib.load("models/model.pkl")              # charge une seule fois


@app.get("/health")
def health():
    return jsonify(statut="ok")


@app.post("/predict")
def predict():
    donnees = request.get_json()                      # le corps JSON de la requete
    X = pd.DataFrame([donnees])[["montant", "anciennete", "freq_24h", "nuit"]]
    p = float(modele.predict_proba(X)[0, 1])
    return jsonify(fraude=p > 0.5, probabilite=round(p, 4))
EOF
```

fastapi_app.py

```
cat > fastapi_app.py << 'EOF'
import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="API de detection de fraude")
modele = joblib.load("models/model.pkl")              # charge une seule fois


class Transaction(BaseModel):                         # le schema d'entree
    montant: float
    anciennete: int
    freq_24h: int
    nuit: int


@app.get("/health")
def health():
    return {"statut": "ok"}


@app.post("/predict")
def predict(t: Transaction):
    X = pd.DataFrame([t.model_dump()])
    p = float(modele.predict_proba(X)[0, 1])
    return {"fraude": p > 0.5, "probabilite": round(p, 4)}
EOF
```

terminal 1

```
flask --app flask_app run --port 5000
```

terminal 2

```
uvicorn fastapi_app:app --port 8000
```

terminal 3

```
python appel.py POST http://localhost:5000/predict
python appel.py POST http://localhost:5000/predict montant=abc
python appel.py POST http://localhost:8000/predict
python appel.py POST http://localhost:8000/predict montant=abc
```

RÉSULTAT ATTENDU

```
200 OK
{"fraude":true,"probabilite":0.8899}

500 INTERNAL SERVER ERROR
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error ...</p>

200 OK
{"fraude":true,"probabilite":0.8899}

422 Unprocessable Content
{"detail":[{"type":"float_parsing","loc":["body","montant"],"msg":"Input should be a valid number, unab
le to parse string as a number","input":"abc"}]}
```

CE QU'IL FAUT VOIR

Pour la même erreur du client, Flask répond 500 (« le serveur a planté ») avec une page HTML, FastAPI répond 422 en JSON et indique le champ fautif (`loc: body, montant`). Le terminal 1 affiche aussi l'avertissement de Flask : « This is a development server. Do not use it in a production deployment ».

Arrêtez les deux serveurs (Ctrl+C dans les terminaux 1 et 2). Pour la suite, on garde FastAPI.

Étape 3 — Partie B · Écrire l'API en six étapes (15 min)

Ce fichier applique les six étapes du cours : (1) le modèle est chargé une seule fois, au démarrage ; (2) le schéma `Transaction` fixe le type et les bornes de chaque champ et refuse les champs inconnus ; (3) le schéma `Prediction` fixe la réponse, avec la version du modèle et un identifiant de requête ; (4) trois routes, dont deux préfixées par `/v1` ; (5) chaque prédiction est journalisée, sans les données du client ; (6) les tests viennent à l'étape 5. Lisez chaque commentaire.

api/main.py

```
cat > api/main.py << 'EOF'
import logging
import time
import uuid

import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, ConfigDict, Field

VERSION_MODELE = "1.0.0"
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
journal = logging.getLogger("fraude-api")
modele = joblib.load("models/model.pkl")              # charge une seule fois, au demarrage
app = FastAPI(title="API de detection de fraude", version=VERSION_MODELE)


class Transaction(BaseModel):                         # ce que l'API accepte
    model_config = ConfigDict(extra="forbid")         # champ inconnu = requete refusee
    montant: float = Field(gt=0, le=1_000_000, description="montant en MAD")
    anciennete: int = Field(ge=0, le=600, description="anciennete du compte, en mois")
    freq_24h: int = Field(ge=0, le=500, description="operations sur les 24 dernieres heures")
    nuit: int = Field(ge=0, le=1, description="1 si l'operation a lieu la nuit")


class Prediction(BaseModel):                          # ce que l'API renvoie
    fraude: bool
    probabilite: float
    version_modele: str
    id_requete: str


@app.get("/health")
```

```
def health():
    return {"statut": "ok", "modele_charge": modele is not None}


@app.get("/v1/model-info")
def model_info():
    return {"version_modele": VERSION_MODELE, "variables": list(modele.feature_names_in_)}


@app.post("/v1/predict", response_model=Prediction)
def predict(t: Transaction):
    debut = time.perf_counter()
    X = pd.DataFrame([t.model_dump()])
    p = float(modele.predict_proba(X)[0, 1])
    rid = uuid.uuid4().hex[:8]
    duree = (time.perf_counter() - debut) * 1000
    journal.info("prediction id=%s version=%s proba=%.4f duree_ms=%.1f",
                 rid, VERSION_MODELE, p, duree)       # aucune donnee personnelle
    return Prediction(fraude=p > 0.5, probabilite=round(p, 4),
                      version_modele=VERSION_MODELE, id_requete=rid)
EOF
```

Ligne	Rôle
Field(gt=0, le=1_000_000)	montant strictement positif, au plus 1 million
extra="forbid"	tout champ non prévu fait refuser la requête
response_model=Prediction	la réponse respecte toujours le même contrat
/v1/...	une future version 2 pourra cohabiter avec la 1
journal.info(...)	id, version, probabilité, durée : aucune donnée client

Étape 4 — Lancer l'API et lire ses réponses (15 min)

`curl -v` montre la requête et la réponse complètes : lignes « > » envoyées par le client, lignes « < » renvoyées par le serveur. Ensuite, chaque appel produit un code HTTP différent. Repérez-les.

terminal 1

```
uvicorn api.main:app --port 8000
```

terminal 2

```
curl -sv http://localhost:8000/health
python appel.py GET http://localhost:8000/health
python appel.py GET http://localhost:8000/v1/model-info
python appel.py POST http://localhost:8000/v1/predict
python appel.py POST http://localhost:8000/v1/predict montant=-5
python appel.py POST http://localhost:8000/v1/predict pays=MA
python appel.py GET http://localhost:8000/v1/predict
python appel.py POST http://localhost:8000/predict
```

RÉSULTAT ATTENDU

```
> GET /health HTTP/1.1
> Host: localhost:8000
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< date: Mon, 09 Nov 2026 15:09:10 GMT
< server: uvicorn
< content-length: 36
< content-type: application/json
<
{"statut":"ok","modele_charge":true}

200 OK
{"statut":"ok","modele_charge":true}
200 OK
{"version_modele":"1.0.0","variables":["montant","anciennete","freq_24h","nuit"]}
200 OK
{"fraude":true,"probabilite":0.8899,"version_modele":"1.0.0","id_requete":"61feed7d"}
422 Unprocessable Content
{"detail":[{"type":"greater_than","loc":["body","montant"],"msg":"Input should be greater than 0","inpu
t":-5,"ctx":{"gt":0.0}}]}
422 Unprocessable Content
{"detail":[{"type":"extra_forbidden","loc":["body","pays"],"msg":"Extra inputs are not permitted","inpu
t":"MA"}]}
405 Method Not Allowed
{"detail":"Method Not Allowed"}
404 Not Found
{"detail":"Not Found"}
```

POINT DE CONTRÔLE

probabilite = 0.8899 pour Mme Benali. 200 = succès ; 422 = données refusées (le message dit pourquoi) ; 405 = bonne adresse mais mauvaise méthode (GET au lieu de POST) ; 404 = adresse inconnue (l'ancienne route /predict n'existe plus).

Ouvrez http://localhost:8000/docs dans un navigateur : c'est le contrat de l'API, généré à partir des schémas. Dépliez POST /v1/predict, « Try it out », saisissez un montant de 0, « Execute » : réponse 422. Le terminal 1 affiche une ligne par appel, et une ligne `prediction id=... proba=0.8899` pour chaque prédiction réussie.

Étape 5 — Écrire les tests automatiques (10 min)

`TestClient` démarre l'application en mémoire : pas besoin de serveur. Les cinq tests couvrent le cas normal et les trois refus du schéma.

tests/test_api.py

```
cat > tests/test_api.py << 'EOF'
from fastapi.testclient import TestClient

from api.main import app

client = TestClient(app)
BENALI = {"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}


def test_health():                          # le service repond et le modele est charge
    r = client.get("/health")
    assert r.status_code == 200 and r.json()["modele_charge"]


def test_prediction_benali():               # cas nominal : la valeur du TP 8
    r = client.post("/v1/predict", json=BENALI)
    assert r.status_code == 200
    assert r.json()["probabilite"] == 0.8899
    assert r.json()["version_modele"] == "1.0.0"


def test_montant_negatif():                 # validation : borne
```

```
    r = client.post("/v1/predict", json=dict(BENALI, montant=-5))
    assert r.status_code == 422


def test_champ_manquant():                  # validation : champ obligatoire
    corps = {k: v for k, v in BENALI.items() if k != "nuit"}
    assert client.post("/v1/predict", json=corps).status_code == 422


def test_champ_inconnu():                   # validation : rien de plus que le contrat
    r = client.post("/v1/predict", json=dict(BENALI, pays="MA"))
    assert r.status_code == 422
EOF
```

terminal 2

```
python -m pytest -q
```

RÉSULTAT ATTENDU

```
.....                                                                    [100%]
5 passed in 1.58s
```

Si ça coince : « ModuleNotFoundError: No module named 'api' » : lancez la commande depuis le dossier `tp10-api`, avec `python -m pytest` et non `pytest` seul.

Étape 6 — Charger le modèle une seule fois : mesurer (10 min)

On crée `api/main_lent.py`, une copie de l'API qui recharge le modèle à chaque requête, puis on envoie 200 requêtes à chacune avec le script de charge. Les deux versions donnent la même probabilité : seule la vitesse change.

tests/charge.py

```
cat > tests/charge.py << 'EOF'
"""Envoie N requetes avec C clients simultanes et affiche la latence."""
import concurrent.futures as cf, statistics, sys, time, requests

URL = sys.argv[1]
N, C = int(sys.argv[2]), int(sys.argv[3])
H = {"X-API-Key": sys.argv[4]} if len(sys.argv) > 4 else {}
CORPS = {"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}

def une():
    t = time.perf_counter()
    requests.post(URL, json=CORPS, headers=H, timeout=30).raise_for_status()
    return (time.perf_counter() - t) * 1000

debut = time.perf_counter()
with cf.ThreadPoolExecutor(C) as ex:
    lat = sorted(ex.map(lambda _: une(), range(N)))
total = time.perf_counter() - debut
print(f"{N} requetes, {C} clients : {N / total:.0f} req/s, "
      f"mediane {statistics.median(lat):.1f} ms, p95 {lat[int(0.95 * N)]:.1f} ms")
EOF
```

api/main_lent.py : la version lente

```
cat > api/main_lent.py << 'EOF'
# VERSION LENTE (etape 6) : copie de api/main.py avec UNE seule difference,
# le modele est recharge a chaque requete dans predict(). Ne pas deployer.
import logging
import time
import uuid

import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, ConfigDict, Field

VERSION_MODELE = "1.0.0"
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
```

```
journal = logging.getLogger("fraude-api")
modele = joblib.load("models/model.pkl")              # charge une seule fois, au demarrage
app = FastAPI(title="API de detection de fraude", version=VERSION_MODELE)


class Transaction(BaseModel):                         # ce que l'API accepte
    model_config = ConfigDict(extra="forbid")         # champ inconnu = requete refusee
    montant: float = Field(gt=0, le=1_000_000, description="montant en MAD")
    anciennete: int = Field(ge=0, le=600, description="anciennete du compte, en mois")
    freq_24h: int = Field(ge=0, le=500, description="operations sur les 24 dernieres heures")
    nuit: int = Field(ge=0, le=1, description="1 si l'operation a lieu la nuit")


class Prediction(BaseModel):                          # ce que l'API renvoie
    fraude: bool
    probabilite: float
    version_modele: str
    id_requete: str


@app.get("/health")
def health():
    return {"statut": "ok", "modele_charge": modele is not None}


@app.get("/v1/model-info")
def model_info():
    return {"version_modele": VERSION_MODELE, "variables": list(modele.feature_names_in_)}


@app.post("/v1/predict", response_model=Prediction)
def predict(t: Transaction):
    debut = time.perf_counter()
    X = pd.DataFrame([t.model_dump()])
    m = joblib.load("models/model.pkl")       # ERREUR VOLONTAIRE : recharge a CHAQUE requete
    p = float(m.predict_proba(X)[0, 1])
    rid = uuid.uuid4().hex[:8]
    duree = (time.perf_counter() - debut) * 1000
    journal.info("prediction id=%s version=%s proba=%.4f duree_ms=%.1f",
                 rid, VERSION_MODELE, p, duree)       # aucune donnee personnelle
    return Prediction(fraude=p > 0.5, probabilite=round(p, 4),
                      version_modele=VERSION_MODELE, id_requete=rid)
EOF
```

Si ça coince : Comparez avec `api/main.py` : seule la ligne `m = joblib.load(...)` dans `predict()` a été ajoutée (plus les deux lignes de commentaire du haut).

terminal 3

```
uvicorn api.main_lent:app --port 8001
```

terminal 2

```
python tests/charge.py http://localhost:8000/v1/predict 200 1
python tests/charge.py http://localhost:8001/v1/predict 200 1
```

RÉSULTAT ATTENDU

```
200 requetes, 1 clients : 47 req/s, mediane 21.5 ms, p95 27.9 ms
200 requetes, 1 clients : 12 req/s, mediane 82.1 ms, p95 102.3 ms

(valeurs variables selon la machine : c'est le rapport entre les deux qui compte)
```

Arrêtez le serveur lent (Ctrl+C dans le terminal 3). Notez les deux médianes : elles vont dans le bilan.

Étape 7 — Servir comme en production : plusieurs processus (10 min)

Avec 20 clients simultanés, un seul processus traite les requêtes à la file. Mesurez avec 1 processus, puis relancez le serveur avec 2 processus (`--workers 2`) et refaites la mesure. On n'utilise plus `--reload` ni le serveur de développement de Flask.

terminal 2 (le serveur du terminal 1 a 1 processus)

```
python tests/charge.py http://localhost:8000/v1/predict 400 20
```

terminal 1 : Ctrl+C, puis

```
uvicorn api.main:app --port 8000 --workers 2
```

terminal 2

```
python tests/charge.py http://localhost:8000/v1/predict 400 20
```

RÉSULTAT ATTENDU

```
1 processus :
400 requetes, 20 clients : 27 req/s, mediane 756.6 ms, p95 968.3 ms
2 processus :
400 requetes, 20 clients : 83 req/s, mediane 216.1 ms, p95 435.9 ms

Terminal 1 :
INFO:     Started parent process [859]
INFO:     Started server process [864]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Started server process [865]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Arrêtez le serveur (Ctrl+C). La partie B est terminée : l'API fonctionne, elle est testée et mesurée. Mais elle est ouverte à tous. C'est l'objet de la partie C.

Étape 8 — Partie C · Préparer le terrain : le brouillon et la version sécurisée (20 min)

Deux versions de l'API vont tourner côte à côte. Le brouillon est ce qu'on écrit trop vite : chaque faille est marquée « FAILLE n », n étant le numéro de la pratique du cours qui la corrige. La version sécurisée est celle du cours. Lisez les deux fichiers en entier.

api/brouillon.py (à ne jamais déployer)

```
cat > api/brouillon.py << 'EOF'
# BROUILLON VOLONTAIREMENT VULNERABLE : NE JAMAIS DEPLOYER
# Chaque faille porte le numero de la bonne pratique du cours qui la corrige.
import logging

import joblib
import pandas as pd
from fastapi import FastAPI, Header, HTTPException

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
journal = logging.getLogger("brouillon")
modele = joblib.load("models/model.pkl")
app = FastAPI(title="Brouillon vulnerable", debug=True)   # FAILLE 6 : erreurs detaillees
CLE_API = "admin"                          # FAILLES 2 et 3 : cle faible, ecrite dans le code


@app.post("/v1/predict")
def predict(donnees: dict, x_api_key: str = Header(None)):  # FAILLE 4 : aucun schema
    if x_api_key != CLE_API:
        raise HTTPException(status_code=401, detail="cle invalide")
    journal.info("cle=%s donnees=%s", x_api_key, donnees)  # FAILLE 8 : secret au journal
    X = pd.DataFrame([donnees])[["montant", "anciennete", "freq_24h", "nuit"]]
    p = float(modele.predict_proba(X)[0, 1])
    return {"fraude": p > 0.5, "probabilite": round(p, 4)}

# FAILLE 1 : lance en HTTP.  FAILLE 5 : aucune limite de debit.
# FAILLE 7 : l'image Docker (Dockerfile.mauvais) tourne en root.
EOF
```

api/main_securisee.py

```
cat > api/main_securisee.py << 'EOF'
import logging
import os
import secrets
import time
import uuid
from collections import defaultdict, deque

import joblib
import pandas as pd
from fastapi import Depends, FastAPI, HTTPException, Security
from fastapi.security import APIKeyHeader
from pydantic import BaseModel, ConfigDict, Field

VERSION_MODELE = "1.0.0"
CLE_API = os.environ["API_KEY"]                       # secret lu dans l'environnement
LIMITE = int(os.getenv("LIMITE_PAR_MINUTE", "60"))    # requetes par minute et par cle
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
journal = logging.getLogger("fraude-api")
modele = joblib.load("models/model.pkl")              # charge une seule fois, au demarrage
app = FastAPI(title="API de detection de fraude", version=VERSION_MODELE)


class Transaction(BaseModel):                         # ce que l'API accepte
    model_config = ConfigDict(extra="forbid")         # champ inconnu = requete refusee
    montant: float = Field(gt=0, le=1_000_000, description="montant en MAD")
    anciennete: int = Field(ge=0, le=600, description="anciennete du compte, en mois")
    freq_24h: int = Field(ge=0, le=500, description="operations sur les 24 dernieres heures")
    nuit: int = Field(ge=0, le=1, description="1 si l'operation a lieu la nuit")


class Prediction(BaseModel):                          # ce que l'API renvoie
    fraude: bool
    probabilite: float
    version_modele: str
    id_requete: str


schema_cle = APIKeyHeader(name="X-API-Key", auto_error=False)   # cle dans l'en-tete X-API-Key


def verifier_cle(cle: str | None = Security(schema_cle)):
    if not cle or not secrets.compare_digest(cle, CLE_API):      # comparaison a temps constant
        raise HTTPException(status_code=401, detail="cle d'API absente ou invalide")
    return cle


appels = defaultdict(deque)                           # heures des derniers appels, par cle


def limiter_debit(cle: str = Depends(verifier_cle)):
    maintenant = time.time()
    fenetre = appels[cle]
    while fenetre and maintenant - fenetre[0] > 60:   # on oublie les appels > 1 minute
        fenetre.popleft()
    if len(fenetre) >= LIMITE:
        raise HTTPException(status_code=429,
                            detail="trop de requetes, reessayez dans une minute")
    fenetre.append(maintenant)


@app.get("/health")
def health():
    return {"statut": "ok", "modele_charge": modele is not None}


@app.get("/v1/model-info", dependencies=[Depends(limiter_debit)])
def model_info():
    return {"version_modele": VERSION_MODELE, "variables": list(modele.feature_names_in_)}
```

```
@app.post("/v1/predict", response_model=Prediction, dependencies=[Depends(limiter_debit)])
def predict(t: Transaction):
    debut = time.perf_counter()
    X = pd.DataFrame([t.model_dump()])
    p = float(modele.predict_proba(X)[0, 1])
    rid = uuid.uuid4().hex[:8]
    duree = (time.perf_counter() - debut) * 1000
    journal.info("prediction id=%s version=%s proba=%.4f duree_ms=%.1f",
                 rid, VERSION_MODELE, p, duree)       # aucune donnee personnelle
    return Prediction(fraude=p > 0.5, probabilite=round(p, 4),
                      version_modele=VERSION_MODELE, id_requete=rid)
EOF
```

Deux fichiers ne doivent jamais quitter votre machine : `.env`, qui contiendra la clé, et le dossier `certs`, qui contiendra le certificat. On les exclut tout de suite de Git et des images Docker.

.gitignore et .dockerignore

```
cat > .gitignore << 'EOF'
.venv
.env
certs
__pycache__
EOF
cat > .dockerignore << 'EOF'
.venv
.env
certs
tests
attaques
**/__pycache__
api/brouillon.py
api/main_lent.py
EOF
```

terminal 3 : créer une clé forte et un certificat HTTPS local

```
python -c "import secrets; open('.env', 'w').write('API_KEY=' + secrets.token_urlsafe(32))"
cat .env
mkdir -p certs
python -m trustme -d certs
```

RÉSULTAT ATTENDU

```
API_KEY=DA78GXP7q_k-5Q9oXeEHOmfwUDBQV_WfTHhJIKEl-CY   (la votre est differente)
Generated a certificate for 'localhost', '127.0.0.1', '::1'
Configure your server to use the following files:
  cert=certs/server.pem
  key=certs/server.key
Configure your client to use the following files:
  cert=certs/client.pem
```

terminal 1 : le brouillon, en HTTP

```
uvicorn api.brouillon:app --port 8001
```

terminal 2 : la version sécurisée, en HTTPS

```
uvicorn api.main_securisee:app --port 8443 --env-file .env \
    --ssl-certfile certs/server.pem --ssl-keyfile certs/server.key
```

terminal 3 : vérifier les deux

```
CLE=$(cut -d= -f2 .env)                # la cle, dans une variable du terminal
python appel.py GET https://localhost:8443/health
python appel.py POST https://localhost:8443/v1/predict --cle $CLE
python appel.py POST http://localhost:8001/v1/predict --cle admin
```

RÉSULTAT ATTENDU

```
Terminal 2 :
INFO:     Loading environment from '.env'
INFO:     Started server process [959]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on https://127.0.0.1:8443 (Press CTRL+C to quit)

Terminal 3 :
200 OK
{"statut":"ok","modele_charge":true}
200 OK
{"fraude":true,"probabilite":0.8899,"version_modele":"1.0.0","id_requete":"9722cd97"}
200 OK
{"fraude":true,"probabilite":0.8899}
```

Port	Rôle
8001	brouillon vulnérable, HTTP (terminal 1)
8443	version sécurisée, HTTPS (terminal 2)
terminal 3	les attaques ; terminal 4 : l'espion (étape 9)
8081 / 8082	conteneurs Docker mauvais / sécurisé (étapes 11 et 15)

Si ça coince : « KeyError: 'API_KEY' » dans le terminal 2 : l'option `--env-file .env` manque, ou le fichier `.env` n'est pas dans le dossier courant. « SSLError ... certificate verify failed » : le dossier `certs` a été recréé après le lancement du serveur ; relancez le terminal 2.

Étape 9 — Pratique 1 · HTTPS : écouter le réseau (15 min)

Attaque. Un espion placé sur le réseau (Wi-Fi public, routeur compromis) lit tout ce qui passe. Le script `espion.py` joue ce rôle : il se place entre le client et le serveur et affiche ce que le client envoie. Lancez-le devant le brouillon, puis devant la version sécurisée.

attaques/espion.py

```
cat > attaques/espion.py << 'EOF'
"""Relais place entre le client et le serveur : affiche ce qui passe sur le reseau.
Usage : python attaques/espion.py PORT_ECOUTE PORT_SERVEUR"""
import socket, sys, threading

ECOUTE, CIBLE = int(sys.argv[1]), int(sys.argv[2])


def lisible(data):                                    # du texte, ou des octets illisibles ?
    return all(32 <= b < 127 or b in (9, 10, 13) for b in data[:200])


def relayer(src, dst, espionner):
    try:
        while data := src.recv(65536):
            if espionner:
                print(f"--- client -> serveur : {len(data)} octets ---")
                if lisible(data):
                    print(data[:400].decode().replace("\r", ""), flush=True)
                else:
                    print(data[:24].hex(" "), "...", flush=True)
            dst.sendall(data)
    except OSError:
        pass
    finally:
        dst.close()


serveur = socket.socket()
serveur.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
serveur.bind(("127.0.0.1", ECOUTE))
serveur.listen()
```

```
serveur.settimeout(1)                                  # Ctrl+C fonctionne aussi sous Windows
print(f"espion en ecoute sur {ECOUTE}, relaie vers {CIBLE}", flush=True)
while True:
    try:
        client, _ = serveur.accept()
    except TimeoutError:
        continue
    client.settimeout(None)
    cible = socket.create_connection(("127.0.0.1", CIBLE))
    threading.Thread(target=relayer, args=(client, cible, True), daemon=True).start()
    threading.Thread(target=relayer, args=(cible, client, False), daemon=True).start()
EOF
```

terminal 4 : espion devant le brouillon

```
python attaques/espion.py 9001 8001
```

terminal 3 : le client passe par l'espion (port 9001)

```
python appel.py POST http://localhost:9001/v1/predict --cle admin
```

RÉSULTAT ATTENDU

```
Terminal 4 :
espion en ecoute sur 9001, relaie vers 8001
--- client -> serveur : 287 octets ---
POST /v1/predict HTTP/1.1
Host: localhost:9001
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
X-API-Key: admin
Content-Length: 61
Content-Type: application/json

{"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}
```

terminal 4 : Ctrl+C, puis espion devant la version sécurisée

```
python attaques/espion.py 9443 8443
```

terminal 3

```
python appel.py POST https://localhost:9443/v1/predict --cle $CLE
```

RÉSULTAT ATTENDU

```
Terminal 4 :
espion en ecoute sur 9443, relaie vers 8443
--- client -> serveur : 1531 octets ---
16 03 01 05 f6 01 00 05 f2 03 03 b7 21 15 e1 c9 8a 08 be 74 90 39 2f 95 ...
--- client -> serveur : 80 octets ---
14 03 03 00 01 01 17 03 03 00 45 cd bf d0 7d c8 88 69 82 e0 2a e5 5a 7f ...
--- client -> serveur : 369 octets ---
17 03 03 01 19 d6 3a d5 fd 84 7f 87 95 2c 32 a4 37 cb fd 6f 3e f6 1c 81 ...
```

CE QUI PROTÈGE

En HTTP, l'espion lit la clé (`X-API-Key: admin`) et le dossier du client, en clair. En HTTPS, il ne voit que des octets chiffrés : le client obtient pourtant sa réponse normalement. Sans HTTPS, même une clé longue et aléatoire est volée au premier appel. Ici, uvicorn fait le chiffrement avec `--ssl-certfile` ; en production, c'est souvent un serveur placé devant l'API (Nginx, passerelle) qui s'en charge, avec un certificat reconnu.

Arrêtez l'espion (Ctrl+C dans le terminal 4).

Étape 10 — Pratique 2 · Clé d'API : deviner la clé (10 min)

Attaque. Sans connaître la clé, on essaie une liste de mots de passe courants : c'est une attaque par dictionnaire.

attaques/deviner_cle.py

```
cat > attaques/deviner_cle.py << 'EOF'
"""Attaque par dictionnaire : essaie des cles d'API courantes."""
import sys

import requests

URL = sys.argv[1]
CORPS = {"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}
DICTIONNAIRE = ["", "test", "1234", "123456", "azerty", "password", "secret",
                "api", "apikey", "cle", "admin", "root", "changeme", "cle-tp-2026"]

for n, cle in enumerate(DICTIONNAIRE, start=1):
    r = requests.post(URL, json=CORPS, headers={"X-API-Key": cle},
                      timeout=5, verify="certs/client.pem")
    print(f"essai {n:2d}  cle={cle!r:14}  -> {r.status_code}")
    if r.status_code == 200:
        print(f"CLE TROUVEE en {n} essais : {cle!r}")
        break
else:
    print(f"echec : aucune des {n} cles n'a fonctionne")
EOF
```

terminal 3

```
python attaques/deviner_cle.py http://localhost:8001/v1/predict
python attaques/deviner_cle.py https://localhost:8443/v1/predict
```

RÉSULTAT ATTENDU

```
essai  1  cle=''              -> 401
essai  2  cle='test'          -> 401
essai  3  cle='1234'          -> 401
essai  4  cle='123456'        -> 401
essai  5  cle='azerty'        -> 401
essai  6  cle='password'      -> 401
essai  7  cle='secret'        -> 401
essai  8  cle='api'           -> 401
essai  9  cle='apikey'        -> 401
essai 10  cle='cle'           -> 401
essai 11  cle='admin'         -> 200
CLE TROUVEE en 11 essais : 'admin'

essai  1  cle=''              -> 401
essai  2  cle='test'          -> 401
...
essai 14  cle='cle-tp-2026'   -> 401
echec : aucune des 14 cles n'a fonctionne
```

CE QUI PROTÈGE

Le brouillon tombe en 11 essais : sa clé est un mot courant. La version sécurisée résiste pour trois raisons : la clé fait 43 caractères tirés au hasard (`secrets.token_urlsafe(32)`) ; elle est lue dans l'environnement (`os.environ["API_KEY"]`) ; `APIKeyHeader` et `secrets.compare_digest` la vérifient à temps constant. La clé convient entre deux services ; pour identifier des personnes, le cours présente JWT et OAuth2.

Étape 11 — Pratique 3 · Secrets : fouiller le code et l'image Docker (15 min)

Attaque. Toute personne qui lit le code (dépôt Git) ou qui récupère l'image Docker cherche les secrets qui y traînent. On construit deux images de la version sécurisée : la « mauvaise » écrit la clé dans le Dockerfile (et tourne en root, ce qui servira à l'étape 15) ; la bonne ne contient aucune clé.

requirements.txt

```
cat > requirements.txt << 'EOF'
fastapi==0.162.2
uvicorn==0.54.0
scikit-learn==1.9.1
pandas==3.1.1
joblib==1.6.0
EOF
```

Dockerfile.mauvais

```
cat > Dockerfile.mauvais << 'EOF'
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY api/ ./api/
COPY models/ ./models/
# FAILLE 3 : la cle est ecrite dans l'image
ENV API_KEY=k3y-ex3mple-2026
# FAILLE 7 : aucune instruction USER, le service tourne en root
EXPOSE 8000
CMD ["uvicorn", "api.main_securisee:app", "--host", "0.0.0.0", "--port", "8000"]
EOF
```

Dockerfile

```
cat > Dockerfile << 'EOF'
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY api/ ./api/
COPY models/ ./models/
# PRATIQUE 7 : un utilisateur sans privileges, jamais root
RUN useradd --create-home app
USER app
EXPOSE 8000
# PRATIQUE 3 : aucune cle dans l'image, elle est fournie au lancement
CMD ["uvicorn", "api.main_securisee:app", "--host", "0.0.0.0", "--port", "8000"]
EOF
```

terminal 3

```
grep -n "CLE_API =" api/brouillon.py api/main_securisee.py
docker build -f Dockerfile.mauvais -t fraude-api:mauvais .
docker build -t fraude-api:securisee .
docker history --no-trunc fraude-api:mauvais | grep API_KEY
docker run --rm fraude-api:mauvais printenv API_KEY
docker history --no-trunc fraude-api:securisee | grep API_KEY
docker run --rm fraude-api:securisee printenv API_KEY
```

RÉSULTAT ATTENDU

```
api/brouillon.py:13:CLE_API = "admin"          # FAILLES 2 et 3 : cle faible, ...
api/main_securisee.py:15:CLE_API = os.environ["API_KEY"]     # secret lu dans ...

Construction de fraude-api:mauvais :
 1 warning found (use docker --debug to expand):
 - SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data
   (ENV "API_KEY") (line 8)

<missing>   3 minutes ago   ENV API_KEY=k3y-ex3mple-2026   0B   buildkit.dockerfile.v0
k3y-ex3mple-2026
(rien pour l'image securisee : aucune ligne, aucune cle)
```

CE QUI PROTÈGE
Une clé écrite dans le code ou dans le Dockerfile se retrouve dans Git et dans chaque copie de l'image : `docker history` et `printenv` la donnent à n'importe qui, et Docker vous prévient dès la construction. La bonne image ne contient ni clé ni `.env` (exclu par `.dockerignore`) : la clé est fournie au lancement, avec `--env-file .env` (étape 15). En production, elle vient d'un coffre à secrets (Vault, coffre du fournisseur cloud).

Étape 12 — Pratique 4 · Validation : envoyer des données piégées (10 min)

Attaque. On envoie six dossiers absurdes ou malformés. Une API qui les accepte renvoie une prédiction sans valeur, ou plante.

attaques/entrees_piegees.py

```
cat > attaques/entrees_piegees.py << 'EOF'
"""Envoie des donnees absurdes ou malformees et affiche la reaction de l'API."""
import sys

import requests

URL, CLE = sys.argv[1], sys.argv[2]
BENALI = {"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}
PIEGES = [("montant negatif", dict(BENALI, montant=-5000)),
          ("nuit = 7", dict(BENALI, nuit=7)),
          ("montant gigantesque", dict(BENALI, montant=1e300)),
          ("champ en plus", dict(BENALI, role="admin")),
          ("montant texte", dict(BENALI, montant="abc")),
          ("champ manquant", {"montant": 1800, "anciennete": 30, "freq_24h": 4})]

for nom, corps in PIEGES:
    r = requests.post(URL, json=corps, headers={"X-API-Key": CLE},
                      timeout=5, verify="certs/client.pem")
    print(f"{nom:20} -> {r.status_code}  {r.text[:56]}".replace("\n", " "))
EOF
```

terminal 3

```
python attaques/entrees_piegees.py http://localhost:8001/v1/predict admin
python attaques/entrees_piegees.py https://localhost:8443/v1/predict $CLE
```

RÉSULTAT ATTENDU

```
Brouillon :
montant negatif      -> 200  {"fraude":false,"probabilite":0.0011}
nuit = 7             -> 200  {"fraude":true,"probabilite":0.8899}
montant gigantesque  -> 500  Traceback (most recent call last):   File "/home/claude/
champ en plus        -> 200  {"fraude":true,"probabilite":0.8899}
montant texte        -> 500  Traceback (most recent call last):   File "/home/claude/
champ manquant       -> 500  Traceback (most recent call last):   File "/home/claude/

Version securisee :
montant negatif      -> 422  {"detail":[{"type":"greater_than","loc":["body","montant
nuit = 7             -> 422  {"detail":[{"type":"less_than_equal","loc":["body","nuit
montant gigantesque  -> 422  {"detail":[{"type":"less_than_equal","loc":["body","mont
champ en plus        -> 422  {"detail":[{"type":"extra_forbidden","loc":["body","role
montant texte        -> 422  {"detail":[{"type":"float_parsing","loc":["body","montan
champ manquant       -> 422  {"detail":[{"type":"missing","loc":["body","nuit"],"msg"
```

CE QUI PROTÈGE
Le brouillon reçoit un simple `dict` : il calcule une probabilité pour un montant négatif ou pour `nuit = 7`, accepte un champ `role` inventé, et plante (500) sur le reste. La version sécurisée refuse tout avec 422, avant même d'appeler le modèle, grâce au schéma `Transaction` : types, bornes `Field(...)` et `extra="forbid"`. Règle du cours associée : ne jamais accepter de fichier pickle venant d'un client, car le charger exécute du code.

Étape 13 — Pratique 5 · Limite de débit : inonder l'API (10 min)

Attaque. On envoie 100 requêtes aussi vite que possible : c'est ainsi qu'on sature un service ou qu'on aspire un modèle en l'interrogeant en masse.

attaques/inonder.py

```
cat > attaques/inonder.py << 'EOF'
"""Envoie N requetes le plus vite possible et compte les codes HTTP recus."""
import collections, sys, time

import requests

URL, CLE, N = sys.argv[1], sys.argv[2], int(sys.argv[3])
CORPS = {"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}
session = requests.Session()
codes = collections.Counter()
debut = time.perf_counter()
for _ in range(N):
    r = session.post(URL, json=CORPS, headers={"X-API-Key": CLE},
                     timeout=5, verify="certs/client.pem")
    codes[r.status_code] += 1
duree = time.perf_counter() - debut
print(f"{N} requetes en {duree:.1f} s")
for code, nombre in sorted(codes.items()):
    print(f"  HTTP {code} : {nombre}")
EOF
```

terminal 3

```
python attaques/inonder.py http://localhost:8001/v1/predict admin 100
python attaques/inonder.py https://localhost:8443/v1/predict $CLE 100
python appel.py POST https://localhost:8443/v1/predict --cle $CLE
```

RÉSULTAT ATTENDU

```
100 requetes en 2.0 s
  HTTP 200 : 100
100 requetes en 1.4 s
  HTTP 200 : 60
  HTTP 429 : 40
429 Too Many Requests
{"detail":"trop de requetes, reessayez dans une minute"}
```

CE QUI PROTÈGE

La fonction `limiter_debit` garde l'heure des derniers appels de chaque clé : au-delà de 60 par minute (`LIMITE_PAR_MINUTE`), la réponse est 429. Si vous avez appelé l'API dans la dernière minute, vous obtiendrez un peu moins de 60 réponses 200. Ce compteur est en mémoire : il est propre à chaque processus et s'efface au redémarrage. En production, la limite est tenue par une passerelle ou un stockage partagé (Redis).

Étape 14 — Pratique 6 · Erreurs discrètes : faire parler le serveur (10 min)

Attaque. On provoque volontairement une erreur et on lit la réponse : elle peut révéler les chemins, le code et les bibliothèques du serveur, autant d'indices pour la suite de l'attaque. Le script résume ce que l'attaquant apprend.

attaques/provoquer_erreur.py

```
cat > attaques/provoquer_erreur.py << 'EOF'
"""Provoque une erreur et resume ce que la reponse revele a un attaquant."""
import re, sys

import requests

URL, CLE = sys.argv[1], sys.argv[2]
corps = {"montant": "abc", "anciennete": 30, "freq_24h": 4, "nuit": 1}
r = requests.post(URL, json=corps, headers={"X-API-Key": CLE},
                  timeout=5, verify="certs/client.pem")
lignes = r.text.splitlines()
print(f"HTTP {r.status_code} - {len(r.text)} caracteres de reponse")
fichiers = [l for l in lignes if l.strip().startswith('File "') and "site-packages" not in l]
for l in fichiers:
```

```
    if "/api/" in l or "\\api\\" in l:
        i = lignes.index(l)
        print("chemin et ligne du code :", l.strip())
        print("instruction fautive     :", lignes[i + 1].strip())
libs = sorted(set(re.findall(r"site-packages[/\\](\w+)", r.text)))
if libs:
    print("bibliotheques visibles  :", ", ".join(libs))
    print("derniere ligne          :", lignes[-1])
else:
    print("reponse                 :", r.text[:150])
EOF
```

terminal 3

```
python attaques/provoquer_erreur.py http://localhost:8001/v1/predict admin
python attaques/provoquer_erreur.py https://localhost:8443/v1/predict $CLE
```

RÉSULTAT ATTENDU

```
HTTP 500 - 5420 caracteres de reponse
chemin et ligne du code : File "/home/etudiant/tp-mlops/tp10-api/api/brouillon.py", line 22, in predict
instruction fautive     : p = float(modele.predict_proba(X)[0, 1])
bibliotheques visibles  : anyio, fastapi, pandas, sklearn, starlette
derniere ligne          : ValueError: could not convert string to float: 'abc'

HTTP 422 - 151 caracteres de reponse
reponse                 : {"detail":[{"type":"float_parsing","loc":["body","montant"],"msg":"Input shou
ld be a valid number, unable to parse string as a number","input":"abc"}]
```

Corrigez maintenant vous-même la faille 6 du brouillon, puis relancez-le (Ctrl+C dans le terminal 1 et même commande qu'à l'étape 8) et rejouez l'attaque.

terminal 3

```
sed -i 's/debug=True/debug=False/' api/brouillon.py
python attaques/provoquer_erreur.py http://localhost:8001/v1/predict admin
```

Si ça coince : sous macOS, `sed -i` demande un argument vide : `sed -i '' 's/debug=True/debug=False/' api/brouillon.py`.

RÉSULTAT ATTENDU

```
HTTP 500 - 21 caracteres de reponse
reponse                 : Internal Server Error
```

CE QUI PROTÈGE

Avec `debug=True`, le brouillon renvoie 5 420 caractères de trace : chemin du fichier, ligne de code, bibliothèques. Corrigé, il ne renvoie que « Internal Server Error » ; le détail reste dans le terminal 1, pour vous seul. La version sécurisée ne va même pas jusqu'à l'erreur : la validation répond 422. Même règle pour Flask : jamais `--debug` en production (capture dans le cours).

Étape 15 — Pratique 7 · Conteneur sans privilèges : prendre la main dans le conteneur (15 min)

Attaque. Si une faille permet d'exécuter des commandes dans le conteneur, l'attaquant obtient les droits de l'utilisateur du service. On simule cette situation avec `docker exec` : peut-il déposer un faux modèle à côté du vrai ? On lance les deux images de l'étape 11.

terminal 3

```
docker run -d --name mauvais -p 8081:8000 fraude-api:mauvais
docker run -d --name securisee -p 8082:8000 --env-file .env fraude-api:securisee
python appel.py POST http://localhost:8082/v1/predict --cle $CLE
docker exec mauvais whoami
docker exec mauvais touch models/modele_pirate.pkl
docker exec mauvais ls -l models
docker exec securisee whoami
docker exec securisee touch models/modele_pirate.pkl
docker exec securisee ls -l models
```

RÉSULTAT ATTENDU

```
2372756dafed
2444df0f3aa2
200 OK
{"fraude":true,"probabilite":0.8899,"version_modele":"1.0.0","id_requete":"f1b4084f"}
root
total 7956
-rw-r--r-- 1 root root 8145273 Nov  9 15:07 model.pkl
-rw-rw-rw- 1 root root       0 Nov  9 15:14 modele_pirate.pkl
app
touch: cannot touch 'models/modele_pirate.pkl': Permission denied
total 7956
-rw-r--r-- 1 root root 8145273 Nov  9 15:07 model.pkl
```

CE QUI PROTÈGE

Le conteneur mauvais tourne en root : l'attaquant écrit où il veut, y compris dans `models/`. Les deux lignes `RUN useradd --create-home app` et `USER app` du bon Dockerfile font tourner le service avec un utilisateur ordinaire : « Permission denied ». Le conteneur sécurisé a reçu sa clé par `--env-file .env` et répond bien 0,8899.

Si ça coince : « port is already allocated » : un autre conteneur occupe 8081 ou 8082 ; `docker rm -f mauvais securisee` puis relancez. « Conflict. The container name is already in use » : même commande.

Étape 16 — Pratique 8 · Journal : repérer les attaques (5 min)

Défense. Relisez les terminaux 1 et 2 : ils contiennent la trace de tout ce que vous venez de faire. Comparez ce qu'y écrit chaque version.

RÉSULTAT ATTENDU

```
Terminal 1 (brouillon) :
2026-11-09 15:15:00,821 INFO cle=admin donnees={'montant': 1800, 'anciennete': 30, 'freq_24h': 4, 'nuit
': 1}
INFO:     127.0.0.1:44998 - "POST /v1/predict HTTP/1.1" 200 OK

Terminal 2 (version securisee) :
2026-11-09 15:10:28,213 INFO prediction id=9722cd97 version=1.0.0 proba=0.8899 duree_ms=42.0
INFO:     127.0.0.1:38314 - "POST /v1/predict HTTP/1.1" 401 Unauthorized
INFO:     127.0.0.1:38416 - "POST /v1/predict HTTP/1.1" 422 Unprocessable Content
INFO:     127.0.0.1:56950 - "POST /v1/predict HTTP/1.1" 429 Too Many Requests
```

CE QUI PROTÈGE

Le journal du brouillon contient la clé et le dossier du client : quiconque lit les journaux (collègues, outil de supervision) les obtient. La version sécurisée n'écrit que l'identifiant, la version, la probabilité et la durée. Ses lignes 401, 422 et 429 racontent l'attaque : 14 essais de clé, des données piégées, puis un débit anormal. Une alerte sur ces codes signale l'attaque pendant qu'elle a lieu.

Étape 17 — Bilan et nettoyage (10 min)

Complétez le bilan avec vos propres mesures et observations, puis arrêtez tout.

terminal 3

```
cat > bilan-tp10.md << 'EOF'
# TP 10 - Bilan

## Mesures (etapes 6 et 7)
| Mesure                          | req/s | mediane | p95 |
|---------------------------------|-------|---------|-----|
| Modele charge une seule fois    |       |         |     |
| Modele recharge a chaque appel  |       |         |     |
| 1 processus, 20 clients         |       |         |     |
| 2 processus, 20 clients         |       |         |     |

## Securite (etapes 9 a 16)
| Pratique            | Attaque            | Brouillon | Version securisee |
|---------------------|--------------------|-----------|-------------------|
```

```
| 1 HTTPS              | espion             |          |                    |
| 2 Cle d'API          | dictionnaire       |          |                    |
| 3 Secrets            | docker history     |          |                    |
| 4 Validation         | entrees piegees    |          |                    |
| 5 Limite de debit    | 100 requetes       |          |                    |
| 6 Erreurs discretes  | montant = abc      |          |                    |
| 7 Sans privileges    | touch dans models/ |          |                    |
| 8 Journal            | lecture des logs   |          |                    |
EOF
docker rm -f mauvais securisee
docker rmi fraude-api:mauvais
```

RÉSULTAT ATTENDU

```
mauvais
securisee
Untagged: fraude-api:mauvais
```

Arrêtez les serveurs des terminaux 1 et 2 (Ctrl+C). Gardez l'image `fraude-api:securisee` et le dossier `tp10-api` : ils serviront aux TP 11 et 12. Ne commitez jamais `.env` ni `certs` : vérifiez avec `git status` qu'ils n'apparaissent pas.

Questions du compte rendu

1. Pour `montant=abc`, Flask a répondu 500 et FastAPI 422 (étape 2). Lequel aide le développeur de l'application cliente, et pourquoi ?
2. Expliquez la différence entre les codes 404, 405 et 422 obtenus à l'étape 4.
3. Donnez vos deux médianes de l'étape 6 et leur rapport. Pourquoi les cinq tests de l'étape 5 passent-ils quand même avec `main_lent.py` ? Quel type de test aurait détecté le problème ?
4. Donnez vos deux débits de l'étape 7. Avec `--workers 2`, la limite de 60 requêtes par minute de l'étape 13 est-elle encore respectée exactement ? Pourquoi ?
5. L'espion de l'étape 9 a lu la clé `admin`. Une clé de 43 caractères aléatoires aurait-elle été protégée en HTTP ? Que faut-il donc toujours combiner ?
6. À l'étape 10, les 14 essais reçoivent 401 avant d'être comptés par `limiter_debit`. Un attaquant peut donc essayer des millions de clés. Proposez une protection.
7. Pourquoi `.env` figure-t-il à la fois dans `.gitignore` et dans `.dockerignore` ? Où placer la clé en production ?
8. Le brouillon a renvoyé une probabilité pour `nuit = 7` (étape 12). Pourquoi est-ce plus dangereux qu'une erreur 500 ?
9. Parmi les 8 pratiques, lesquelles sont assurées par le code de l'API, et lesquelles par la façon de la déployer (serveur, conteneur, réseau) ?

Livrables

- Le dossier `tp10-api` commité dans Git, sans `.env`, `certs` ni `.venv`
- Le fichier `bilan-tp10.md` complété (mesures et tableau de sécurité)
- Captures : réponses de Flask et FastAPI à `montant=abc` (étape 2), espion en HTTP et en HTTPS (étape 9), dictionnaire sur les deux versions (étape 10), `whoami` et `touch` dans les deux conteneurs (étape 15)
- Le compte rendu avec les réponses aux questions

Barème

Critère	Points
Flask et FastAPI comparés, API écrite et interrogée, codes HTTP expliqués (étapes 1 à 4)	4
Tests qui passent, mesures de chargement et de processus relevées (étapes 5 à 7)	3
Terrain préparé, HTTPS et clé d'API démontrés (étapes 8 à 10)	4
Secrets dans l'image, validation, limite de débit, erreurs discrètes (étapes 11 à 14)	4
Conteneur sans privilèges et lecture du journal (étapes 15 et 16)	2

Bilan complété et réponses aux questions	3
Total	20